

Project 5 and Linux perf

Principles, Events, Counters, and Core Commands

Yuhao LIU

April 16, 2026

Agenda

- 1 Overview
- 2 Principle
- 3 Events and Counters
- 4 Core Commands
- 5 References

Project 5 overview

- This project asks you to optimize several provided C++ kernels under a fixed benchmark framework and explain why the optimizations improve performance on real hardware.
- It is a **team project**: each group has **no more than two students**, and the final report should state how the score is allocated within the team.
- All official compilation, correctness testing, timing, and profiling must be done on the **remote course server**, not on personal laptops.
- Students need to keep the implementations correct, improve performance across tasks such as cache optimization and data structure optimization, and use `perf` to find bottlenecks.
- The final report should clearly present server-side results and discuss the optimization ideas used.

What Is perf?

- perf is the standard Linux performance analysis toolkit.
- It helps us answer questions like:
 - Where is the program spending time?
 - Is the bottleneck CPU, memory, or branching behavior?
 - Which function or line should we optimize first?
- It works for quick measurement (`perf stat`) and deeper profiling (`perf record/report/annotate`).
- The main idea: measure real program behavior instead of guessing.

Why Use perf?

- Lightweight and already available on most Linux systems.
- Can use both hardware information and kernel software events.
- Scales from a single command to whole-system profiling.
- Especially useful when optimizing compute kernels, server code, and hot loops.

Basic Principle of perf

- ① The CPU and kernel expose measurable events.
- ② perf asks the system to watch selected events while the program runs.
- ③ The system maintains counters, such as total cycles or cache misses.
- ④ If a counter overflows at a chosen period, a sample is taken.
- ⑤ Samples are aggregated into reports that show hotspots.

Simple View

Program execution → events happen → counters increase → samples are collected → perf shows where the cost is

Where the Data Comes From

Hardware Side

- CPU cycles
- Instructions retired
- Branches / branch misses
- Cache references / misses
- ...

Kernel / Software Side

- Context switches
- Page faults
- CPU migrations
- Tracepoints and kernel events
- ...

Key Idea

perf combines low-level counters with symbol information so we can map machine activity back to functions and source code.

Events and Counters

- **Event:** something we want to observe.
 - Examples: cycles, instructions, cache-misses, page-faults
- **Counter:** a value maintained by hardware or the kernel that increases when the event happens.
- **Derived metrics:** useful ratios computed from counters.
 - $IPC \approx \text{instructions} / \text{cycles}$
 - $\text{Cache miss rate} \approx \text{cache-misses} / \text{cache-references}$
- Different CPUs support different hardware events, so available counters may vary by machine.

Counter Mode vs Sampling Mode

Counter Mode

- Total counts for the full run
- Low overhead
- Good for first diagnosis
- Main command: `perf stat`

Sampling Mode

- Records representative execution points
- Shows hotspots and call stacks
- Better for “where” questions
- Main commands: `perf record/report`

Rule of Thumb

Start with counters to see *what* is wrong, then use sampling to find *where* it is wrong.

perf stat: Overall Measurement

```
csc3060@csc3060-proj5:~/CSC3060_Project5_Code_Optimization/build$ perf stat ./single_bench
ReLU: vector length=1024000

Running Benchmarks...
-----
Benchmark      Status      Nanoseconds
-----
ReLU (Naive)    PASSED      1479482 ns (SLOW)

Performance counter stats for './single_bench':

    2420.28 nsec task-clock           #    1.000 CPUs utilized
         3      context-switches      #    1.240 /sec
         0      cpu-migrations        #    0.000 /sec
    33902      page-faults            #   14.007 K/sec
  780224322    cycles                  #    2.893 GHz
 20904972006   instructions            #    3.00  insn per cycle
 2451424282    branches                #    1.013 G/sec
   1378562     branch-misses           #    0.06% of all branches

    2.42060374 seconds time elapsed

    2.336576000 seconds user
    0.084020000 seconds sys

csc3060@csc3060-proj5:~/CSC3060_Project5_Code_Optimization/build$
```

`perf stat ./single_bench`

`perf stat -e cycles,instructions,cache-misses ./single_bench`

- Reports aggregate counts for one execution.
- Good for checking runtime, cycles, instructions, IPC, and miss behavior.
- Often the best first step before deeper profiling.
- Useful for before/after comparison when testing an optimization.

perf record: Collect Samples

```
csc3000@csc3000-proj5: /CSC3000-Projects/Code/Optimization/buil$ perf record -g -F 999 ./single_bench
WARNING: Kernel address maps (/proc/{kallsyms,modules}) are restricted,
check /proc/sys/kernel/kptr_restrict and /proc/sys/kernel/perf_event_paranoid.

Samples in kernel functions may not be resolved if a suitable vmlinux
file is not found in the buildid cache or in the vmlinux path.

Samples in kernel modules won't be resolved at all.

If some relocation was applied (e.g. kexec) symbols may be misresolved
even with a suitable vmlinux or kallsyms file.

Couldn't record kernel reference relocation symbol
Symbol resolution may be skewed if relocation was used (e.g. kexec).
Check /proc/kallsyms-permission or run as root.
    ReLU: vector length=1024000

Running Benchmarks...
-----
Benchmark      Status      Nanoseconds
-----
ReLU (Naive)    PASSED      1467191 ns (SLOW)
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.238 MB perf.data (2409 samples) ]
csc3000@csc3000-proj5: /CSC3000-Projects/Code/Optimization/buil$
```

perf record ./single_bench

perf record -g -F 999 ./single_bench

- Records profiling data into perf.data.
- -g adds call graph information, which is very helpful for hotspot analysis.
- Can profile a command directly or attach to a running process.
- Use this when you need to know which functions are actually hot.

perf report: Read the Hotspots

Samples: 2K of event 'cycles:P', Event count (approx.): 7089653948				
Children	Self	Command	Shared Object	Symbol
99.85%	0.00%	single bench	single bench	[.] _start
99.81%	0.00%	single bench	libc.so.6	[.] libc_start_main
99.74%	0.00%	single bench	libc.so.6	[.] 0x00007f8229eb5dca
94.74%	0.00%	single bench	single bench	[.] main
78.67%	44.80%	single bench	single bench	[.] flush_cache
27.90%	27.90%	single bench	single bench	[.] std::vector<int, std::allocator<int>*>::operator[](unsig
19.78%	19.78%	single bench	single bench	[.] std::vector<int, std::allocator<int>*>::size() const
3.26%	0.00%	single bench	single bench	[.] _GLOBAL__sub_I_main
2.26%	0.00%	single bench	single bench	[.] _Static_initialization_and_destruction_0()
2.26%	0.00%	single bench	single bench	[.] std::vector<int, std::allocator<int>*>::vector(unsigned
3.26%	0.00%	single bench	single bench	[.] std::vector<int, std::allocator<int>*>::N::default_init
2.26%	0.00%	single bench	single bench	[.] int* std:: uninitialized default n a<int*, unsigned lo
3.26%	0.00%	single bench	single bench	[.] int* std:: uninitialized default n<int*, unsigned long
3.26%	0.00%	single bench	single bench	[.] int* std:: uninitialized default n<true>:: uninit d
3.26%	0.00%	single bench	single bench	[.] int* std::fill_n<int*, unsigned long, int>(int*, unsigh
3.26%	2.34%	single bench	single bench	[.] qmex:: enable_if<std:: is_scalar<int>:: value,
3.26%	0.00%	single bench	single bench	[.] int* std:: fill n a<int*, unsigned long, int>(int*, un
3.26%	0.00%	single bench	single bench	[.] void std:: fill a<int*, int>(int*, int*, int const&)
1.87%	0.00%	single bench	[unknown]	[.] 0xffffffffa000c0
1.54%	0.00%	single bench	[unknown]	[.] 0xffffffffa3d2337
1.52%	0.00%	single bench	libc.so.6	[.] 0x00007f822a02c43a
2.46%	0.00%	single bench	[unknown]	[.] 0xffffffffa0a0499
1.42%	0.00%	single bench	[unknown]	[.] 0xffffffffa090a998
1.42%	0.00%	single bench	[unknown]	[.] 0xffffffffa090ccd5
1.42%	0.00%	single bench	[unknown]	[.] 0xffffffffa090acd5

perf report

- Opens the recorded data and summarizes where samples landed.
- Common columns include overhead, command, shared object, and symbol.
- Functions with high overhead are the first optimization targets.
- With call graphs enabled, we can see the callchains (press “e”).
- More help? Press “?”

A Simple Workflow

- 1 Run `perf stat` to measure the overall behavior.
- 2 Run `perf record -g` to collect hotspot samples.
- 3 Open `perf report` to find expensive functions.
- 4 Use `perf annotate` to inspect the most expensive code region.
- 5 Modify the program and repeat with the same workload.

Practical Tip

Always profile with a realistic input and compare results on the same machine.

- Linux kernel perf wiki: <https://perf.wiki.kernel.org/>
- Linux manual pages: `man perf`, `man perf-stat`, `man perf-record`, `man perf-report`, `man perf-annotate`
<https://man7.org/linux/man-pages/man1/perf.1.html>
- Brendan Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd edition
- Brendan Gregg, perf examples: <https://www.brendangregg.com/>

Thank You

Questions?